

Agentic Artificial Intelligence

Chapter 03: LLMs, RAG & Tool Use

How Language Models Become Agents

Prof. Dr. Thomas Clemen

BT7, R. 11.83 | thomas.clemen@haw-hamburg.de



Review + Questions

Semester Plan

Chapter 1. Agents and their Fundamentals

Chapter 2. Perception, Memory, Planning & Learning

Chapter 3. LLMs, RAG & Tool Use

Chapter 4. Multi-Agent Systems and MARL

Chapter 5. Agentic AI Engineering

Chapter 6. Safety & Guardrails

The Three Gaps of LLMs

Three experiments – three failures



"What did I order last week?"

No long-term memory



"Book the cheapest flight HAM → BER on Monday."

No ability to act



"Compare, evaluate, and book the best option."

No reliable planning

Three Gaps → Three Solutions

Gap		Solution		Cognitive Layer
No long-term memory	→	RAG	→	Memory
No ability to act	→	Tool Use	→	Action
No reliable planning	→	ReAct / CoT	→	Planning



Central thesis: An LLM alone is not an agent. But LLM + Memory (RAG) + Action (Tool Use) + structured Planning (ReAct/CoT) = an agent that realizes the cognitive architecture from Ch. 01/02.

PART 1

LLMs – The Reasoning Engine



What they can do – and what they cannot

What an LLM Brings to the Table



Language Understanding & Generation

High-level comprehension and production of natural language



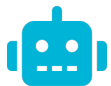
Broad World Knowledge

Semantic memory – but frozen at training cutoff



Reasoning Ability

Inference, analogies, pattern recognition across domains



Flexibility

Can handle almost any text-based task without retraining

Think of the LLM as a brilliant consultant who gives great advice – but never picks up the phone.

What an LLM Does Not Bring

Capability	Status	Cognitive Layer
Long-term memory	✗ None beyond context window	Memory
Up-to-date knowledge	✗ Frozen at training cutoff	Memory (Semantic)
Acting in the world	✗ Text output only, no side effects	Action
Reliable planning	⚠ Can formulate plans, not execute them	Planning
Persistence across sessions	✗ Every call is a fresh start	Memory (Episodic)

Reflection: In the agent taxonomy from Ch. 01, a pure LLM is at best a limited Goal-Based Agent – it can understand goals and formulate plans, but it cannot act and it forgets everything.

Architecture of an LLM Call

From an agent perspective – not an ML perspective

INPUT (Prompt)

- System Prompt
- User Message
- Chat History
- RAG Context

Perception Layer



Reasoning
Engine

OUTPUT

- Text response
- JSON / structured data
- Tool call
- Plan

Action Layer

Key insight: Everything the agent knows must be in the prompt. The context window is its entire working memory. The output is text – to become an action, a mechanism must translate text into function calls.

PART 2



Retrieval-Augmented Generation

Closing the Memory Gap

The Problem RAG Solves

1 Knowledge Gap

The LLM only knows what was in its training data (cutoff date). Company documents, current data, personal preferences? Missing.

2 Capacity Gap

Even if you could put everything in the prompt – the context window is finite and the LLM loses track (Lost-in-the-Middle).



RAG: The Architectural Answer

Instead of giving the LLM everything at once, give it exactly the right information at the right time.

RAG Architecture

OFFLINE: INDEXING

Documents



Chunking



Embedding



Vector DB

ONLINE: RETRIEVAL → AUGMENTATION → GENERATION

1. RETRIEVAL

Query → Embedding
→ Vector search
→ Top-k chunks



2. AUGMENTATION

Query + Retrieved
chunks combined
into augmented prompt



3. GENERATION

Augmented prompt
→ LLM
→ Grounded answer

Semantic Search vs. Keyword Search

Keyword Search (traditional)

Query: "side effects of Aspirin"

- Matches documents containing the exact words
- "Aspirin" AND "side effects"

✗ Misses: "Acetylsalicylic acid can cause gastric bleeding"

Semantic Search (RAG)

Query: "side effects of Aspirin"

- Matches documents by meaning (vector similarity)
- Similar vectors = similar meaning

✓ Finds: "Acetylsalicylic acid can cause gastric bleeding"

"Google searches for words. RAG searches for meaning."

RAG in the Cognitive Architecture

Mapping RAG to memory types from Chapter 02

Cognitive Layer	Without RAG	With RAG
Memory (Semantic)	Only training weights (frozen)	+ Vector DB (updatable, extensible)
Memory (Episodic)	No memory across sessions	+ Chat log as searchable knowledge
Memory (Working)	Context window (hard limit)	Context window, intelligently filled
Perception	Only the current prompt	Prompt + relevant external info

RAG is essentially an external long-term memory with a search function. Instead of cramming everything into the prompt, we retrieve only what's relevant.

Limitations of RAG

Garbage In, Garbage Out

Incorrect, outdated, or contradictory documents lead to wrong answers

The Chunking Problem

Too small = lost context. Too large = overwhelms the context window

Retrieval Errors

Semantic search isn't perfect. "How tall is the bridge?" may find bridge architecture instead

Hallucination Despite Context

The LLM can still hallucinate even with correctly retrieved context

Latency & Cost

Every RAG call adds a vector DB query – more time, more compute

→ Preview Ch. 06: RAG can increase trustworthiness – or open new attack vectors (Document Injection)

PART 3

Tool Use & Function Calling



Closing the Action Gap

The Action Gap

Recall from Ch. 02: Three dimensions of Action – Capability, Granularity, Irreversibility

The problem:

A pure LLM's only capability is text output. It cannot book a flight, send an email, query a database, or interact with any external system. Its only action is generating characters.



Tool Use

gives the LLM "hands"

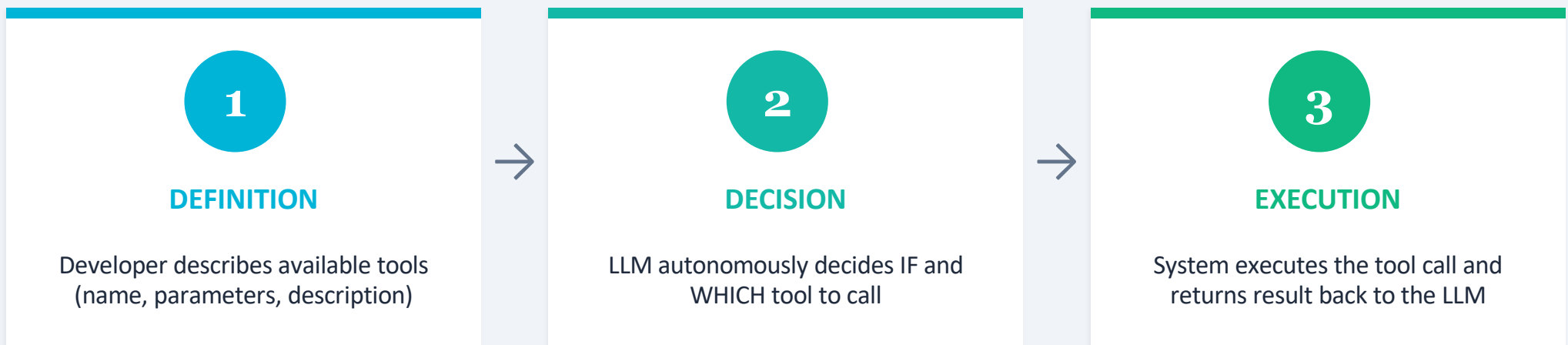


RAG

gives the LLM "memory"

Together, they form an agent.

How Function Calling Works



Key: The LLM doesn't just answer questions – it decides which tool to use, with what parameters, and when. The developer defines the toolbox; the LLM decides how to use it.

Example: Travel Agent with Tool Use

User

"Book cheapest flight HAM→BER Monday"

LLM decides

Tool call: `search_flights("HAM", "BER", "2026-02-23")`

Tool returns

`[{Eurowings, €49, 07:15}, {Lufthansa, €89, 09:30}]`

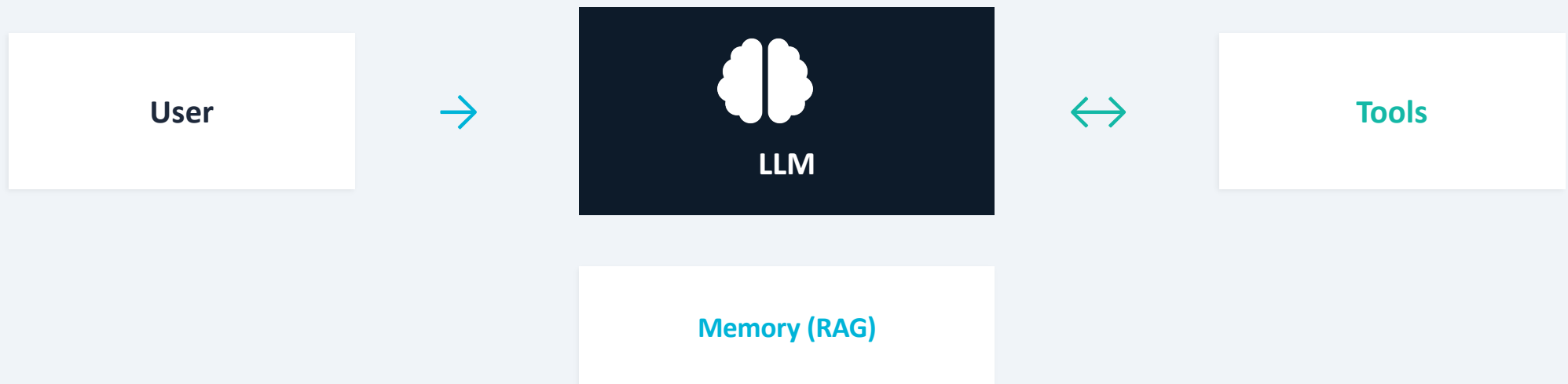
LLM responds

"Cheapest: Eurowings 07:15 for €49. Shall I book?"

The LLM acts as orchestrator: at each step, it decides which tool it needs, calls it, processes the result, and decides whether to continue or respond.
© 2026 Prof. Dr. Th. Clemen, HAW Hamburg

The Tool-Use Loop

Multi-step example: Full flight booking workflow



- ① `search_flights("HAM", "BER", "2026-02-23")`
- ② `get_user_preferences("flight")`
- ③ `check_seat_availability("EW123", "window")`
- ④ Ask user for confirmation
- ⑤ `book_flight("EW123")`

Tool Use in the Cognitive Architecture

Cognitive Layer	Without Tool Use	With Tool Use
Action	Text output only	API calls, DB queries, web search, code execution
Perception	Text input only	+ Structured data from APIs, files, sensors
Planning	Plans verbally, never executes	Plans AND executes (tool call = action)
Memory	Only context window	+ DB access as tool (read/write)

Tool Use doesn't just extend Action – it improves Perception (agent can read APIs), Memory (agent can store/retrieve data), and Planning (plans become executable). It's the single strongest lever to turn an LLM into an agent.

Risks of Tool Use

Irreversibility

Sending emails, placing orders, deleting files – these actions cannot be undone. The LLM must know when to ask instead of act.

Tooling Errors

What happens when the API returns an error? The LLM must handle failures gracefully.

Security

Tools give the LLM access to the real world. A poorly defined tool can be dangerous.

Cost

Every tool call costs time (latency) and potentially money (API costs).

→ **Ch. 05:** Error handling, retries, fallbacks → **Ch. 06:** Giving an agent a tool means giving it power. The question is not just 'Can it use it?' but 'Should it be allowed to?'

PART 4

Agentic Patterns

CoT, ReAct & Structured Output



Chain-of-Thought (CoT)

Making the reasoning process explicit, step by step

Without CoT

"How many tennis balls fit in a school bus?"

"Approximately 500,000."

← *Guessed, no reasoning*

With CoT

1. Bus volume: $\sim 2.5 \times 2 \times 10 = 50 \text{m}^3$
2. Tennis ball: $\sim 6.7 \text{cm} \rightarrow V \approx 0.000157 \text{m}^3$
3. Packing density $\sim 64\%$:
 $50 \times 0.64 / 0.000157 \approx 204,000$
→ **Approximately 200,000 tennis balls**

Connection to cognitive science: CoT is essentially deliberative planning as text generation. The agent simulates its own thinking before acting – exactly what distinguishes a Goal-Based Agent from a Simple Reflex Agent (Ch. 01).

ReAct – Reasoning + Acting

The dominant pattern for LLM-based agents (Yao et al., 2023)

Thought: I need to search for flight options first.

Action: `search_flights("Hamburg", "Berlin", "2026-02-23")`

Observation: [Eurowings €49 07:15, Lufthansa €89 09:30, ...]

Thought: Cheapest is Eurowings €49. Let me check user preferences first.

Action: `get_user_preferences("flight")`

: (loop continues until task is complete)

■ Thought ■ Action ■ Observation

ReAct vs. Pure LLM Call

Property	Pure LLM Call	ReAct Agent
Planning	One-shot, upfront	Iterative, step by step
Observation	None (only the prompt)	Real data from tools
Error correction	Not possible	Reacts to unexpected results
Transparency	Black box	Every thought step traceable
Execution	Describes what it would do	Actually does it

On the planning spectrum from Ch. 02, ReAct sits between rule-based and pure LLM-based planning. It uses the flexibility of the LLM but structures the process through the Thought → Action → Observation cycle.

Structured Output

The bridge between the language world and the API world

The Problem

LLMs generate natural language. Tool calls require valid JSON. Without reliable structured output, tool use breaks down.

Solutions

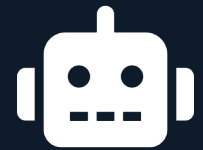
- JSON Mode – API enforces JSON output
- Function Calling – API handles structuring
- Schema Validation – Validate & retry on error

Structured Output is the bridge between the language world of the LLM and the API world of the tools. Without reliable JSON, no tool call works – and without tool calls, the agent cannot act.

PART 5

Integration

The Cognitive Architecture – Realized



The LLM-Based Agent – Complete Architecture

PERCEPTION

User Input (Prompt) · Tool Responses (API data) · RAG Context (relevant docs)



MEMORY

Working: Context Window · Semantic: Vector DB (RAG) · Episodic: Chat History · Buffer: Current Prompt



PLANNING

Chain-of-Thought (explicit reasoning) · ReAct (Think → Act → Observe) · Task Decomposition



ACTION

Tool Calls (APIs, DBs, Web) · Code Execution · Text Output to User

The Travel Agent – Across Three Chapters

Layer	Ch. 01 (Abstract)	Ch. 02 (Mechanisms)	Ch. 03 (Realization)
Perception	PEAS: User prefs, search results	Raw data → Pre-processing → State	Prompt + RAG context + API responses
Memory	"Has access to calendar"	Working (context), Semantic (knowledge)	Context window + Vector DB + Chat log
Planning	"Compares options"	Deliberative, hierarchical decomp.	ReAct loop: Thought → Act → Obs.
Action	PEAS: Initiate booking	Capability, granularity, irreversibility	Function calls: search(), book()

From abstract → mechanisms → concrete technology. The same agent, increasingly realized.

Counterexample: MARS LaserTag Agent

Not every agent needs an LLM – architecture must match the problem

Layer	LLM-Based Travel Agent	MARS LaserTag Agent
Perception	RAG + API calls	Explore() method (direct sensing)
Memory	Vector DB + Context Window	Few variables (Health, Ammo)
Planning	ReAct / CoT (LLM-based)	Rule-based (If-Then)
Action	Function Calls (APIs)	Shoot(), Kneel(), Hide()

Recall Ch. 01: ABM agents are deliberately simple (exploring emergence). Agentic AI agents are deliberately complex (solving tasks). The LaserTag agent works perfectly with simple rules and minimal memory – because the environment allows it. Architecture follows problem, not available technology.

Key Takeaways

An LLM alone is not an agent.

LLM + RAG + Tools + structured Planning = Agent

RAG closes the Memory gap.

External memory with semantic search

Tool Use closes the Action gap.

The LLM gets "hands" to interact with the world

ReAct closes the Planning gap.

Iterative thinking and acting in alternation

Architecture determines behavior.

What Ch. 01 described abstractly is now realized with concrete technologies

Preview: Chapter 04

Multi-Agent Systems

We now have a single agent that can perceive, remember, plan, and act.

But what happens when multiple such agents need to collaborate?

Who coordinates?

Who decides in case of conflict?

How is cooperation organized?

Imagine your travel agent is three agents: one searches flights, one negotiates prices, one checks calendar conflicts. Who gives the orders? Who has the final say?

References & Further Reading

Lewis et al. (2020): "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"

Yao et al. (2023): "ReAct: Synergizing Reasoning and Acting in Language Models"

Schick et al. (2023): "Toolformer: Language Models Can Teach Themselves to Use Tools"

Wei et al. (2022): "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models"

OpenAI Cookbook: Function Calling

Anthropic Docs: Tool Use